Informe de seguridad

Auditoría OWASP Mobile Top 10 (2024) de MackeUp AI

Materia	Programación Móvil
Profesor	Alex Vacca
Integrantes	[Completar con los nombres del grupo]
Repositorio	github.com/Karquin06/ProjectMake-UpAi — rama `feature/seguridad`
Versión auditada	Commit `28fc2f7` (12 de noviembre de 2026)
Entrega	16 de noviembre de 2026

Las evidencias de este informe ocultan llaves, tokens y datos personales. Los reportes completos de las herramientas están en `auditoria/evidencia/`, que no se sube al repositorio.

1. Resumen ejecutivo

MackeUp AI es una app Flutter de asesoría de imagen: analiza la colorimetría con una selfie, recomienda maquillaje y ropa, tiene un simulador de maquillaje en realidad aumentada, un asistente con IA y un módulo de marcas. El backend es Firebase (Authentication, Cloud Firestore, Cloud Storage, Cloud Functions y Cloud Messaging).

Se revisaron los diez riesgos del OWASP Mobile Top 10 (2024) con cinco técnicas: búsqueda de secretos con gitleaks, análisis de dependencias, revisión del manifiesto y los permisos, análisis del APK de release y revisión del código de las reglas y de las Cloud Functions. La base es sólida: las reglas y las funciones validan la sesión, los roles y las entradas en el servidor, y no hay contraseñas ni tokens guardados en el dispositivo. El problema más grave es de configuración: **el APK de release se firma con una llave publicada en el repositorio público**.

Resultado de las correcciones: se cerraron 8 de los 9 hallazgos, cada uno en su propio commit, y se verificaron repitiendo la misma prueba (secciones 6 y 7). H-05 queda parcial con 7 avisos moderados residuales en los SDK de Google y H-06 (App Check) queda como mejora pendiente.

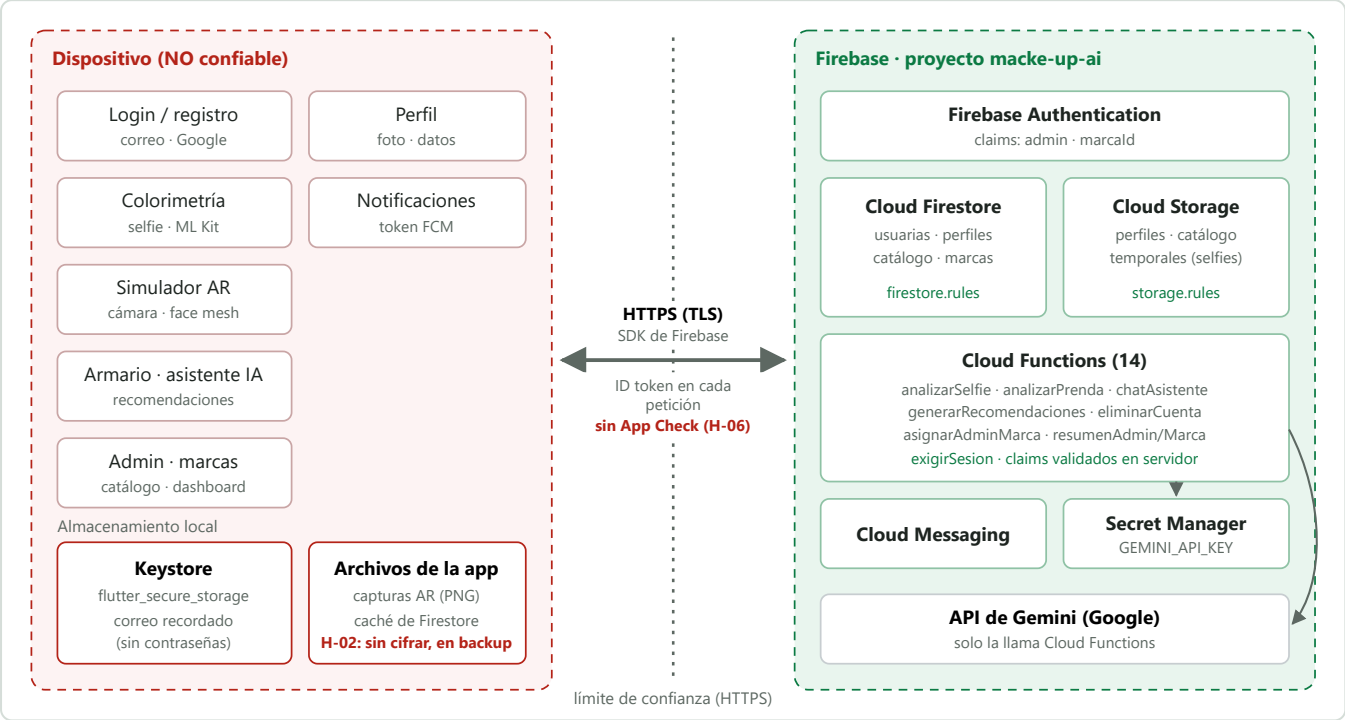
9	1	5	3
hallazgos registrados	de severidad alta	de severidad media	de severidad baja

ID	Riesgo	Estado	Hallazgos
M1	Improper Credential Usage	**No cumple**	H-01
M2	Inadequate Supply Chain Security	**No cumple**	H-05
M3	Insecure Authentication/Authorization	**No cumple**	H-03, H-06
M4	Insufficient Input/Output Validation	**Cumple**	—
M5	Insecure Communication	**Cumple**	— (falta la prueba dinámica)
M6	Inadequate Privacy Controls	**No cumple**	H-07, H-08
M7	Insufficient Binary Protections	**No cumple**	H-04
M8	Security Misconfiguration	**No cumple**	H-01, H-07
M9	Insecure Data Storage	**No cumple**	H-02, H-09
M10	Insufficient Cryptography	**Cumple**	—

2. Modelo de amenazas

Regla base: el dispositivo y la app instalada no son de confianza. Toda decisión de permisos se valida en las reglas de Firebase o en Cloud Functions.

2.1 Diagrama de la app y flujo de datos



2.2 Datos sensibles

Dato	Dónde se guarda	En tránsito	Sensibilidad
Correo y contraseña	Firebase Auth; en el dispositivo solo el correo (Keystore)	HTTPS a Auth	Alta
Sesión (ID token / refresh token)	Manejado por el SDK de Firebase Auth	Cabeceras a Functions	Alta
Selfie y fotos del rostro (AR)	Temporal de análisis (se borra); capturas AR en archivos de la app	HTTPS a Storage y Functions	Alta biométrico
Rol de administradora / admin de marca	Custom claims del token de Auth	—	Alta
Perfil de colorimetría	Firestore `perfiles_colorimetria`	HTTPS	Media
Foto de perfil, conversaciones con la IA, armario	Storage y Firestore	HTTPS	Media
Token de notificaciones (FCM)	Firestore `usuarias/{uid}`	HTTPS	Baja

2.3 Amenazas

ID	Amenaza	Quién	OWASP	Control esperado
A-01	Distribuir un APK modificado firmado con la misma llave que la app real, porque la llave está en el repositorio público.	Cualquiera que clone el repo	M1, M8	Llave de release privada; rotar la expuesta (H-01)

ID	Amenaza	Quién	OWASP	Control esperado
A-02	Una marca desactivada o una ex-admin de marca sigue modificando o borrando productos del catálogo.	Ex-administradora de una marca	M3	Reglas que revisen `activa` y revocación de sesión (H-03)
A-03	Recuperar las fotos del rostro del simulador AR desde el respaldo de Google o una copia del teléfono.	Persona con acceso a la cuenta o al dispositivo	M9, M6	Excluir del respaldo y cifrar (H-02)
A-04	Llamar a las Cloud Functions con un script, sin la app, para gastar la cuota de Gemini o falsear las estadísticas de marcas.	Atacante externo con cuentas en masa	M3	App Check con Play Integrity y llaves restringidas (H-06)

Ya mitigado en el diseño: modificar la app para entrar al panel de administración no sirve, porque las reglas y las funciones validan el claim `admin` en el servidor.

3. Metodología

La auditoría se hizo sobre el commit `28fc2f7` con scripts reproducibles en `auditoria/scripts/`. Cada script guarda su salida con fecha y commit, para repetir la misma prueba después de cada corrección (Fase 4).

Técnica	Herramienta / script	Riesgos	Resultado
Secretos en código e historial	gitleaks 8.30.1 — `escanear_secretos.ps1`	M1, M10	81 commits y todas las ramas; 6 coincidencias en el historial, 3 en los archivos actuales
Dependencias	`flutter pub outdated`, `npm audit` — `revisar_dependencias.ps1`	M2	10 vulnerabilidades en `functions/`
Manifiesto y permisos	`revisar_manifiesto.ps1`	M5, M6, M8	Fuente y fusionado de release, `Info.plist`
Análisis del APK de release	`apkanalyzer`, `apksigner` — `construir_apk.ps1`	M7, M8	Firma, `debuggable`, símbolos en `libapp.so`
Revisión de código	Lectura de `firestore.rules`, `storage.rules` y 14 Cloud Functions	M3, M4, M9	Autorización, validación de entradas, almacenamiento
Logs y tráfico en ejecución	Logcat y mitmproxy 12.3.1 en el emulador `auditoria_api35`	M5, M9	Pendiente: se ejecutan en la Fase 4

> **Reglas seguidas:** solo se probó nuestra app y nuestro proyecto de Firebase. Ninguna evidencia muestra llaves completas, tokens ni datos personales: las llaves aparecen recortadas como `AIzaSyAl…`.

4. Revisión del OWASP Mobile Top 10

M1 · Improper Credential Usage — No cumple

gitleaks marca como `gcp-api-key` las llaves de cliente de Firebase en `lib/firebase_options.dart` y `android/app/google-services.json` (4 distintas en el historial por el cambio de proyecto). Son identificadores públicos por diseño: el acceso lo controlan las reglas, así que se aceptan como falso positivo, con la condición de restringirlas (H-06). La llave de Gemini está en Secret Manager (`defineSecret`), `functions/.env` solo tiene el nombre del modelo en todo su historial y las cuentas de servicio están en `.gitignore`. **Sí hay una credencial expuesta:** `debug.keystore` y su contraseña, que además firman el release (H-01).

```
gitleaks git --log-opts=--all   →  81 commits scanned · leaks found: 6
  4  gcp-api-key  lib/firebase_options.dart
  2  gcp-api-key  android/app/google-services.json
```

M2 · Inadequate Supply Chain Security — No cumple

`npm audit` en `functions/` reporta 10 vulnerabilidades transitivas con arreglo disponible (H-05). En Flutter hay versiones mayores pendientes de Firebase y ML Kit, sin avisos de seguridad publicados, y un `dependency_overrides` que fija `permission_handler_android` en 13.0.1.

M3 · Insecure Authentication/Authorization — No cumple

Cumple: las 11 funciones `onCall` exigen sesión y las de administración verifican el claim `admin` en el servidor. Las reglas limitan cada colección a su dueña, impiden asignarse `rol` o `marcaId` y cierran las colecciones internas. `eliminarCuenta` exige reautenticación reciente. **Falla:** una marca desactivada conserva permisos de escritura y quitar el rol no revoca la sesión (H-03). Además, no hay App Check (H-06).

M4 · Insufficient Input/Output Validation — Cumple

Cada función valida tipos, largos y listas blancas: mensajes de chat de hasta 500 caracteres y 10 de historial, límite de mensajes por minuto, `limite` entre 1 y 50, identificadores sin `/` y rutas de temporales con patrón estricto que evita `..`. Las reglas validan tipos y enumeraciones y los formularios validan URL y correo. No hay deep links. Observación: el historial del chat lo envía el cliente; el impacto se limita a la propia conversación.

M5 · Insecure Communication — Cumple

No se define `usesCleartextTraffic` ni `network_security_config`. Con `targetSdk` 36 el tráfico en claro está bloqueado por defecto y todo va por HTTPS a servicios de Google. `Info.plist` no tiene `NSAllowsArbitraryLoads`. La prueba con mitmproxy queda para la Fase 4. No hay certificate pinning (mejora opcional).

M6 · Inadequate Privacy Controls — No cumple

Cumple: la colorimetría pide consentimiento; las selfies se analizan en un temporal que se borra y las que van a Functions se eliminan en menos de una hora; existe la eliminación de cuenta. **Falla:** se piden permisos de galería innecesarios (H-07) y las fotos de perfil son legibles por cualquier usuaria (H-08).

M7 · Insufficient Binary Protections — No cumple

El código Dart se compila sin `--obfuscate`: nombres de clases y métodos quedan legibles en `libapp.so` (H-04). No hay verificación de integridad del dispositivo ni de la app (H-06).

```
APK release (commit 28fc2f7) — lib/arm64-v8a/libapp.so
  rutas "package:mackeupai/...dart" legibles:  209
  nombres: CredencialRepository · CapturasArRepository · asignarAdminMarca · correoRecordado
classes.dex: 989 clases renombradas por R8 (Logcat: t7.b.call()(r8-map-id-...))
apkanalyzer manifest debuggable  →  false
```

M8 · Security Misconfiguration — No cumple

El release no es `debuggable` y el único componente exportado es `MainActivity` (el lanzador). Pero el release se firma con la llave de depuración pública (H-01) y pide permisos de más (H-07). En iOS faltan `NSCameraUsageDescription` y `NSPhotoLibraryUsageDescription` (observación, la entrega es Android).

M9 · Insecure Data Storage — No cumple

Cumple: `flutter_secure_storage` (Keystore) solo guarda el correo recordado y la marca del onboarding; nunca contraseñas ni tokens. **Falla:** las capturas AR del rostro se guardan sin cifrar y entran en el respaldo de Android (H-02), y 16 `debugPrint` escriben en Logcat en release (H-09).

M10 · Insufficient Cryptography — Cumple

No se usan MD5, SHA-1, DES ni ECB, ni hay llaves embebidas. El único hash propio es SHA-256, que seudonimiza a la usuaria en las estadísticas del servidor. El cifrado local lo hace el Keystore.

5. Hallazgos

Alta H-01 · Release firmado con una llave publicada

M1 · M8

Descripción

El APK de release se firma con `android/app/debug.keystore`, versionado en un repositorio **público** con su contraseña escrita en Gradle. Cualquiera puede firmar un APK con la misma firma y distribuirlo como actualización por fuera de la tienda. Su huella está registrada en Firebase para el inicio de sesión con Google.

Evidencia

```
android/app/build.gradle.kts
31  signingConfigs { getByName("debug") {
32      storeFile = file("debug.keystore")
33      storePassword = "android"  …  }}
40  buildTypes { release {
44      signingConfig = signingConfigs.getByName("debug") }}

$ git ls-files | grep keystore   →  android/app/debug.keystore   (commit 6830141)
```

Corrección

Llave de release fuera del repo (`key.properties` + `.jks` en `.gitignore`) y registrar su SHA en Firebase. Sacar `debug.keystore` del repo y rotar la huella registrada.

Media H-02 · Fotos del rostro sin cifrar y dentro del respaldo

M9

Descripción

Las capturas del simulador AR se guardan en PNG en `getApplicationDocumentsDirectory()/resultados_ar/`. La carpeta de cada cuenta es el uid en base64, lo que no las protege. `android:allowBackup` no está definido (por defecto `true`): las fotos y la caché de Firestore se copian al respaldo de Google y a la transferencia entre teléfonos.

Evidencia

```
revisar_manifiesto.ps1 → <application>
  allowBackup            (sin definir)
  fullBackupContent      (sin definir)
  dataExtractionRules    (sin definir)
lib/data/repositories/capturas_ar_repository.dart L10-33  →
File('…/resultados_ar/$cuenta/$id.png')
```

Corrección

`android:allowBackup="false"` o reglas de extracción que excluyan `resultados_ar/` y la base de Firestore. Opcional: cifrar las capturas con una llave en Keystore.

Media H-03 · Permisos de marca que no se retiran

M3

Descripción

Al desactivar una marca, `resumenMarca` le bloquea el panel, pero las reglas solo miran el claim `marcaId`: su admin sigue pudiendo crear, editar y borrar productos e imágenes. Además, `asignarAdminMarca` cambia el claim sin revocar la sesión, así que quien pierde el rol lo conserva hasta que expire su token (hasta 1 hora).

Evidencia

```
firestore.rules L131-139  allow create/update/delete … esAdminMarca(marcaId)   // no revisa marcas/{id}.activa
storage.rules  L35-40     request.auth.token.get('marcaId', null) == marcaId
functions/src/marcas.ts L367  auth.setCustomUserClaims(uid, claims)    // sin revokeRefreshTokens
```

Corrección

Exigir `get(/…/marcas/$(marcaId)).data.activa == true` en Firestore y Storage para la admin de marca, y llamar a `auth.revokeRefreshTokens(uid)` al quitar o cambiar la marca.

Media H-04 · Código Dart sin ofuscar en release

M7

Descripción

El release se compila sin `--obfuscate --split-debug-info`. Los nombres de clases, métodos y rutas del proyecto quedan legibles en `libapp.so`, lo que facilita entender la lógica de roles y las llamadas al backend.

Evidencia

```
flutter build apk --release            (sin --obfuscate)
grep -a -o "package:mackeupai/.*.dart" libapp.so | sort -u | wc -l   →  209
grep -a -c "CredencialRepository" libapp.so                          →  1
```

Corrección

`flutter build apk --release --obfuscate --split-debug-info=build/simbolos` (opción `-Ofuscado` del script), con los símbolos guardados fuera del repo.

Media H-05 · Dependencias con vulnerabilidades conocidas

M2

Descripción

`npm audit` en `functions/`: 10 vulnerabilidades transitivas, todas con arreglo.

Evidencia

```
body-parser 2.0.0 - 2.3.0   Severity: high      (denegación de servicio)
minimatch, qs, uuid, google-gax …   Severity: moderate
10 vulnerabilities (1 low, 8 moderate, 1 high)  ·  fix available via `npm audit fix`
```

Corrección

`npm audit fix`, actualizar `firebase-admin` (14.5.0 → 14.5.1), correr `npm test` y repetir el análisis.

Media H-06 · Backend sin App Check

M3 · M1

Descripción

Functions, Firestore y Storage aceptan peticiones de cualquier cliente que use la configuración pública, que está en un repositorio público. Un script puede crear cuentas y llamar a `chatAsistente` (gasta cuota de Gemini) o falsear eventos de marcas. No se pudo verificar si las llaves tienen restricciones en Google Cloud.

Evidencia

```
grep -r "app_check|AppCheck|enforceAppCheck" pubspec.yaml lib functions/src   → sin resultados
```

Corrección

`firebase_app_check` con Play Integrity, `enforceAppCheck: true` en las funciones y activación en Firestore y Storage; restringir las llaves por paquete, SHA y API. (Mejora opcional del PDF.)

Baja H-07 · Permisos que la app no usa

M6 · M8

Descripción

Se declaran y se piden `READ_MEDIA_IMAGES` y `READ_EXTERNAL_STORAGE`, aunque `image_picker` usa el selector del sistema, que no los necesita en Android 13+. El APK final además declara `RECORD_AUDIO`, `WRITE_EXTERNAL_STORAGE`, `USE_BIOMETRIC` y `USE_FINGERPRINT` (de plugins) sin usarlos: las cámaras usan `enableAudio: false`.

Evidencia

```
AndroidManifest.xml L6-9   READ_MEDIA_IMAGES · READ_EXTERNAL_STORAGE (maxSdk 32)
permisos_service.dart L34-39   [Permission.photos, Permission.storage]
apkanalyzer manifest print (APK release) → + RECORD_AUDIO · WRITE_EXTERNAL_STORAGE · USE_BIOMETRIC · USE_FINGERPRINT
```

Corrección

Quitar esos permisos (los de plugins con `tools:node="remove"`) y la solicitud de `Permission.photos`.

Baja H-08 · Fotos de perfil visibles para todas

M6

Descripción

Cualquier usuaria con sesión puede leer `perfiles/{uid}/foto.jpg` de otra; la app no tiene ninguna función que lo requiera.

Evidencia

```
storage.rules L18-19   match /perfiles/{uid}/{archivo} { allow read: if logueada(); }
```

Corrección

`allow read: if esDuena(uid) || esAdmin();`

Baja H-09 · Logs de depuración activos en release

M9

Descripción

16 `debugPrint` sin `kDebugMode` escriben en Logcat en release: errores de Firebase Messaging y del detector AR, stack traces y metadatos de la cámara. No se vieron tokens ni datos personales.

Evidencia

```
simulador_ar_view_model.dart L267  debugPrint('AR error real: $mensaje');
                             L268  if (stack != null)
debugPrint(stack.toString());
notificaciones_provider.dart L51   debugPrint('No se pudo guardar el token FCM:
$e');
… 16 en total (simulador AR, maquillaje AR, malla facial, notificaciones)
```

Corrección

Envolver en `if (kDebugMode)` o usar un logger que se apague en release; verificar con `logcat.ps1 -Release`.

6. Correcciones

Se corrigieron **8 de los 9 hallazgos**, empezando por el de mayor severidad, cada uno en su propio commit de la rama `feature/seguridad` con su ID en el mensaje. Antes de cada commit pasaron `flutter analyze`, `flutter test` (326 tests) y `npm test` (51 tests).

ID	Mejora obligatoria	Cambio	Commit
H-01	Ningún secreto en el repositorio	El release se firma con una llave propia (`android/key.properties` + `.jks`, ignorados por Git). Gradle se detiene si falta, en vez de usar la de depuración. Script `crear_llave_release.ps1`.	`e85e1aa`
H-03	Permisos validados en el servidor	Firestore y Storage exigen marca activa y correo en `marcas/{id}.admins`; `asignarAdminMarca` revoca los refresh tokens; `resumenMarca` revisa la lista. Test nuevo.	`bca8a32`
H-02	Datos sensibles guardados de forma segura	`allowBackup="false"`, `fullBackupContent="false"` y reglas de extracción que excluyen todo (capturas AR y caché de Firestore).	`868452b`
H-04	Release con R8 y ofuscado	Build de release con `--obfuscate --split-debug-info` por defecto; símbolos fuera del repo.	`4d1386e`
H-09	Release sin logs sensibles	`debugPrint` pasa a ser una función vacía en release (`lib/main.dart`).	`f88462c`
H-05	Dependencias sin vulnerabilidades	`npm audit fix` y `firebase-admin` 14.5.1: se elimina la vulnerabilidad alta.	`0e8c8f5`
H-07	Mínimo privilegio	Se quitan `READ_MEDIA_IMAGES`, `READ_EXTERNAL_STORAGE` y, con `tools:node="remove"`, `RECORD_AUDIO`, `WRITE_EXTERNAL_STORAGE`, `USE_BIOMETRIC` y `USE_FINGERPRINT`.	`2738357`
H-08	Mínimo privilegio	Fotos de perfil legibles solo por su dueña o la admin.	`32401ed`
H-06	Opcional: integridad	No se implementó: App Check exige registrar la app en Play Integrity desde la consola de Firebase.	—

> **Pendiente del equipo:** publicar las reglas y funciones (`firebase deploy --only firestore:rules,storage,functions`), crear la llave de publicación definitiva y registrar sus huellas SHA en Firebase. La llave usada en la verificación es de prueba y solo existe en el equipo donde se generó.

7. Verificación (antes y después)

Se repitió la misma prueba de cada hallazgo sobre la versión auditada (`28fc2f7`) y sobre la rama corregida. Un hallazgo solo se cierra si la prueba repetida ya no lo reproduce.

ID	Prueba	Antes	Después	Cerrado
H-01	`apksigner verify --print-certs`	`CN=Android Debug, O=MakeUpAi` (llave del repo)	`CN=MackeUp AI, O=UNICESAR` (llave propia, fuera del repo)	Sí
H-02	`dumpsys package` en el emulador	`flags=[… ALLOW_BACKUP]`	`flags=[HAS_CODE ALLOW_CLEAR_USER_DATA]`, sin `ALLOW_BACKUP`	Sí

ID	Prueba	Antes	Después	Cerrado
H-03	`probar_reglas.ps1` (emulador, proyecto demo)	Ex-admin y marca desactivada escriben: 200 (3 fallas)	Ambos reciben 403; la admin vigente sigue con 200	Sí
H-04	Rutas `package:mackeupai/…` en `libapp.so`	209 rutas y nombres de clases	0 rutas; sin nombres de clases (solo textos literales)	Sí
H-05	`npm audit`	10 (1 alta)	7 moderadas, 0 altas (residual en SDK de Google)	Parcial
H-07	Permisos del APK (`apkanalyzer`)	14, con micrófono y almacenamiento	8: sin micrófono, almacenamiento ni biometría	Sí
H-08	`probar_reglas.ps1`	Otra usuaria lee la foto: 200	Otra usuaria: 403; la dueña: 200	Sí
H-09	Código y Logcat del release	16 `debugPrint` activos	`debugPrint` vacío en release	Sí

```
probar_reglas.ps1 -Commit 28fc2f7   →  Resultado: 4 caso(s) no cumplen
probar_reglas.ps1  (corregido)      →  Resultado: todas las reglas se comportan como se espera
npm audit  antes: 10 vulnerabilities (1 low, 8 moderate, 1 high)   después: 7 moderate
```

8. Referencias

- OWASP Mobile Top 10 (2024) — owasp.org/www-project-mobile-top-10
- OWASP MASVS — mas.owasp.org/MASVS
- OWASP MASTG 2.0.0 — mas.owasp.org/MASTG
- MASTG-KNOW-0036: Shared Preferences
- Firebase: “Learn about using and managing API keys for Firebase” y documentación de App Check
- Android Developers: Auto Backup, Photo Picker y firma de apps